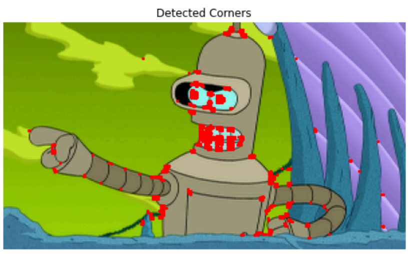
Classical_CV

1. Harris Corner Detection
    1. Gradient computation
        1. Compute the gradients I_x and I_y using the Sobel operators:
        2. $I_x = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}$ and $\begin{pmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{pmatrix}$
        3. Convolve the grayscale image with I_x and I_y to obtain the gradient images I_x and I_y.
    2. Structure Tensor calculation
        1. Compute the elements of the structure tensor:
        2. I_x^2, I_y^2, and $I_x \cdot I_y$
        3. These computations involve element-wise multiplication of I_z and I_y with themselves.
    3. Harris response calculation
        1. For each pixel (i, j), compute the elements of the structure tensor within a local window of size $N \times N$. Compute the sums of squares of I_x and I_y within the window to obtain S_{xy}. Compute the determinant and trace of structure tensor:
        2. $\det(M) = S_{xx} \times S_{yy} - S_{xy}^2$, $\mathrm{Tr}(M) = S_{xx} + S_{yy}$
        3. Compute the Harris response (R) using the formula:
        4. $R = \det(M) - k \times (\mathrm{Tr}(M))^2$
        5. where k is an empirically chosen constant.
    4. Thresholding and Non-maximum Suppression
        1. Apply a threshold to R to identify potential corner candidates. Perform non-maximum suppression to retain only local maxima as corner points.
    5. Visualization
        1. Plot the original image with detected corner points marked.



    6.
    7.

-- coding: utf-8 --

""""

Created on Tue Feb 17 22:10:18 2026

@author: reina
""""

from scipy.ndimage import convolve
from skimage import color
from skimage import io
import numpy as np
import cv2
import matplotlib.pyplot as plt

def harris_corner_detection(image, threshold=0.01, window_size=3, k=0.04):
# Convert the image to grayscale

gray = color.rgb2gray(image)

```
# Compute gradients using Sobel operators
Ix = convolve(gray, np.array([[-1, 0, 1], [-2, 0, 2], [-1, 0, 1]]))
Iy = convolve(gray, np.array([[-1, -2, -1], [0, 0, 0], [1, 2, 1]]))

# Compute elements of the structure tensor
Ix2 = Ix * Ix
Iy2 = Iy * Iy
Ixy = Ix * Iy

# Compute Harris response
height, width = gray.shape
R = np.zeros((height, width))
offset = window_size // 2
for i in range(offset, height - offset):
    for j in range(offset, width - offset):
        Sxx = np.sum(Ix2[i - offset:i + offset + 1, j - offset:j + offset + 1])
        Syy = np.sum(Iy2[i - offset:i + offset + 1, j - offset:j + offset + 1])
        Sxy = np.sum(Ixy[i - offset:i + offset + 1, j - offset:j + offset + 1])
        det = Sxx * Syy - Sxy**2
        trace = Sxx + Syy
        R[i, j] = det - k * trace**2

# Thresholding
corners = np.zeros_like(R)
corners[R > threshold * np.max(R)] = 1
# Perform non-maximum suppression
#corners = non_maximum_suppression(corners)

return corners
```

def non_maximum_suppression(corners):
# Define 8-neighbor offsets
offsets = [(i, j) for i in [-1, 0, 1] for j in [-1, 0, 1] if (i != 0 or j != 0)]

```
# Create a copy of the corners matrix
suppressed_corners = corners.copy()

# Iterate over each pixel in the corners matrix
height, width = corners.shape
for i in range(height):
    for j in range(width):
        # Check if the current pixel is a corner
        if corners[i, j] > 0:
            # Calculate the sum of the values of the neighboring pixels
            neighbor_sum = 0
            for offset_i, offset_j in offsets:
                ni, nj = i + offset_i, j + offset_j
                if ni >= 0 and ni < height and nj >= 0 and nj < width:
                    neighbor_sum += corners[ni, nj]

            # If the sum of neighboring pixels is less than 4, keep the current pixel as a corner
            if neighbor_sum < 4:
                suppressed_corners[i, j] = 1
            else:
                suppressed_corners[i, j] = 0
```

```
return suppressed_corners
```

Load the example image

img = cv2.imread('bender.png')
img = cv2.resize(img, (300, 169), interpolation=cv2.INTER_CUBIC)
#plt .imshow(img, cmap='gray')

Perform Harris corner detection

corners = harris_corner_detection(img)

Visualize the detected corners

plt.figure(figsize=(8, 6))
plt.imshow(img)
plt.scatter(np.nonzero(corners)[1], np.nonzero(corners)[0], color='r', s=5)
plt.title('Detected Corners')
plt.axis('off')
plt.show()
```

2. SIFT
1. People say 'SIFT' but there are two parts to SIFT [1].
1. an interest point detector.
2. a region descriptor.
1. They are independent, many people use the region descriptor without looking for the points.
2. SIFT summary [2]:
1. Extract a 16x16 patch around detected keypoint.
2. Compute the gradients and apply a Gaussian weighting function.
3. Divide the window into a 4x4 grid of cells.
4. Compute gradient direction histograms over 8 directions in each cell.
5. Concatenate the histograms over 8 directions in each cell.
6. Concatenate the histograms to obtain 128 dimensional feature vector.
7. Normalize to unit length.

| | Rotation Invariant | Scale Invariant | Affine Invariant | Repeatability | Localization Accuracy | Robustness | Efficiency |
|---|---|---|---|---|---|---|---|
| Haris | x | | | +++ | +++ | ++ | ++ |
| Shi-Tomasi | x | | | +++ | +++ | ++ | ++ |
| FAST | x | x | | ++ | ++ | ++ | ++++ |
| SIFT | x | x | x | +++ | ++ | +++ | + |
| SURF | x | x | x | +++ | ++ | ++ | ++ |

Image Courtesy of Davide Scaramuzza et. al 2012.

Local features: main components [3]
3. Detection: Identify the interest points.
4. Description: Extract vector feature descriptor surrounding each interest point.
5. Matching: Determine correspondence between descriptors in two views.

References:

1. CSE576 Udub Linda Shapiro
2. Trym Vegard Haavardsholm, UNIK4690
```

3. CSIE 5429 3D-DLCV, National Taiwan University, Spring 2021